

■ 강의명: CSCI103

■ 주차: Lecture 5

■ 교수명: UIUC Student

■ 목적: 데이터 레이크, 데이터 웨어하우스, 그리고 이 둘의 장점을 결합한 데이터 레이크하우스 아키텍처를 심층적으로 다룹니다. 특히 델타 레이크(Delta Lake)가 데이터 레이크의 한계를 극복하고 레이크하우스의 핵심 구성 요소로 기능하는 방식에 초점을 맞춥니다. 스파크(Spark)의 내부 동작 원리, 데이터 처리 성능 최적화 기법, 데이터 거버넌스, 그리고 최신 데이터 아키텍처 패러다임인 데이터 메시(Data Mesh)와 데이터 패브릭(Data Fabric)에 대한 이해를 목표로 합니다. 비전공자도 쉽게 이해할 수 있도록 각 개념의 필요성, 직관적인 비유, 기술적 설명 및 실제 코드 예시를 포함하여 시험 대비 및 실무 활용에 필요한 지식을 제공합니다.

Contents

1 개요	2
2 용어 정리	2
3 핵심 개념 및 원리	3
4 데이터 아키텍처 패러다임: 데이터 메시 vs. 데이터 패브릭	12
5 절차 및 방법: Databricks Jobs 워크플로우 오케스트레이션	14
6 실습: 델타 레이크 기능 활용	16
7 FAQ (자주 묻는 질문)	20
8 빠르게 훑어보기 (1페이지 요약)	21
9 체크리스트: 학습 및 과제 대비	23

1 개요

본 문서는 CSCI103 Lecture 5 강의 자료를 바탕으로 데이터 레이크, 데이터 웨어하우스, 그리고 이 둘의 장점을 결합한 데이터 레이크하우스 아키텍처를 심층적으로 다룹니다. 특히 델타 레이크(Delta Lake)가 데이터 레이크의 한계를 극복하고 레이크하우스의 핵심 구성 요소로 기능하는 방식에 초점을 맞춥니다. 스파크(Spark)의 내부 동작 원리, 데이터 처리 성능 최적화 기법, 데이터 거버넌스, 그리고 최신 데이터 아키텍처 패러다임인 데이터 메시(Data Mesh)와 데이터 패브릭(Data Fabric)에 대한 이해를 목표로 합니다. 비전공자도 쉽게 이해할 수 있도록 각 개념의 필요성, 직관적인 비유, 기술적 설명 및 실제 코드 예시를 포함하여 시험 대비 및 실무 활용에 필요한 지식을 제공합니다.

2 용어 정리

핵심 용어집

용어	쉬운 설명	원어	비고
데이터 사일로	각 부서가 데이터를 따로 보관하여 공유가 어려운 상태	Data Silo	고객 360도 뷰 방해
데이터 레이크	정형/비정형 데이터를 모두 저장하는 거대한 저장소	Data Lake	저렴하고 유연하나, 관리 부실 시 문제 발생
데이터 swamp	관리되지 않아 쓸모없어진 데이터 레이크	Data Swamp	데이터 레이크의 실패 사례
데이터 웨어하우스	정형 데이터를 분석하기 좋게 구조화하여 저장하는 곳	Data Warehouse	빠르고 정교하나, 비싸고 유연성 부족
레이크하우스	데이터 레이크의 유연성과 웨어하우스의 안정성 결합	Lakehouse	델타 레이크가 핵심 기술
DAG	작업 순서를 나타내는 방향성 없는 그래프	Directed Acyclic Graph	스파크 작업의 실행 계획
스파크 잡	스파크에서 실행되는 하나의 큰 작업 단위	Spark Job	여러 스테이지와 태스크로 구성
스테이지	스파크 잡 내에서 서플 없이 실행되는 태스크 묶음	Stage	
태스크	스파크에서 워커 노드에서 실행되는 가장 작은 작업 단위	Task	
스필	메모리 부족 시 디스크에 임시 데이터를 쓰는 현상	Spill	성능 저하의 주범
서플	워커 노드 간 데이터 교환이 필요한 작업	Shuffle	네트워크 비용 발생, 성능 저하
데이터 스큐	특정 워커 노드에 데이터가 몰려 작업 부하가 불균형한 현상	Data Skew	일부 노드만 바쁘고 나머지는 대기
스트래글러	스큐로 인해 작업이 지연되는 노드	Straggler	
작은 파일 문제	너무 많은 작은 파일로 인해 I/O 오버헤드가 커지는 문제	Small Files Problem	파일 압축(compaction)으로 해결
스키마 온 라이트	데이터를 저장하기 전에 엄격한 스키마 정의 필요	Schema on Write	데이터베이스, 데이터 웨어하우스 방식
스키마 온 리드	데이터를 저장할 때는 스키마에 구애받지 않고, 읽을 때 스키마 정의	Schema on Read	데이터 레이크 방식, 소비가 어려움
메달리온 아키텍처	데이터 품질에 따라 데이터를 계층화하는 방식	Medallion Architecture	브론즈, 실버, 골드 레이어로 구성
브론즈 레이어	원본 데이터를 그대로 저장하는 영역	Bronze Layer	원시 데이터, 변경 불가
실버 레이어	정제, 필터링, 보강된 데이터를 저장하는 영역	Silver Layer	신뢰할 수 있는 데이터
골드 레이어	특정 비즈니스 목적에 맞게 집계된 데이터를 저장하는 영역	Gold Layer	분석 및 AI/ML 모델링에 사용
델타 레이크	데이터 레이크 위에 ACID 트랜잭션 등 기능 추가한 프로토콜	Delta Lake	파게이(Parquet) 포맷 기반
ACID 트랜잭션	데이터 무결성을 보장하는 속성 (원자성, 일관성, 고립성, 지속성)	ACID Transactions	관계형 DB의 핵심 기능
타임 트래블	데이터의 과거 특정 시점 상태로 돌아가 조회/복구하는 기능	Time Travel	데이터 변경 이력 관리
스키마 진화	데이터 스키마 변경(컬럼 추가 등)을 유연하게 처리하는 기능	Schema Evolution	데이터 레이크의 유연성 유지
데이터 스킵핑	쿼리 시 불필요한 데이터 블록을 건너뛰어 성능 향상	Data Skipping	메타데이터 활용
Z-오더링	데이터 저장 순서를 최적화하여 쿼리 성능 향상	Z-ordering	다차원 클러스터링
오토로더	클라우드 스토리지에서 파일 변경을 감지하여 자동 로드	Autoloader	증분 로드, 스키마 추론/진화 지원
카피 인투	클라우드 스토리지에서 데이터를 한 번에 대량 로드하는 SQL 명령	Copy Into	멥등성(Idempotent) 보장
DLT	데이터 파이프라인을 선언적으로 정의하는 프레임워크	Delta Live Tables	데이터 품질 검사, 자동 복구 기능 내장
유니티 카탈로그	데이터 및 AI 자산에 대한 통합 거버넌스 솔루션	Unity Catalog	메타데이터 관리, 접근 제어
포톤	스파크 워크로드를 위한 고성능 쿼리 엔진	Photon	C++로 재작성된 스파크 엔진
서버리스	사용자가 인프라를 직접 관리할 필요 없이 컴퓨팅 자원 사용	Serverless	Databricks의 최신 컴퓨팅 모델
크론 표현식	특정 시간/주기작업을 예약하는 표준 형식	Cron Expression	Unix 기반 스케줄링
파일 트리거	특정 파일이 도착했을 때 작업을 시작하는 방식	File Trigger	이벤트 기반 워크플로우
수리 후 실행	실패한 워크플로우를 실패 지점부터 재시작하는 기능	Repair and Run	시간 및 컴퓨팅 비용 절약
데이터 메시	데이터를 제품처럼 취급하고 도메인별로 소유/관리하는 분산 아키텍처	Data Mesh	조직적, 문화적 변화 강조
데이터 패브릭	다양한 데이터 소스를 통합하여 단일 환경처럼 제공하는 기술 아키텍처	Data Fabric	기술 중심, 통합 및 자동화 강조

3 핵심 개념 및 원리

1. 데이터 아키텍처의 진화: 사일로, 레이크, 웨어하우스, 레이크하우스

데이터 관리 방식은 시대의 요구에 따라 끊임없이 진화해왔습니다. 초기에는 각 부서가 데이터를 개별적으로 관리하는 '사일로' 형태였으나, 데이터의 양과 종류가 폭발적으로 증가하면서 더 효율적이고 통합적인 접근 방식이 필요해졌습니다.

데이터 사일로 (Data Silo)

한 줄 핵심 요약: 각 부서나 시스템이 데이터를 독립적으로 보관하여 공유와 통합이 어려운 상태입니다. **직관적 비유:** 각 부서가 자기만의 금고에 데이터를 넣어두고 열쇠를 공유하지 않는 것과 같습니다. **기술적 설명:** 전통적인 데이터 마트(Data Mart)나 데이터 웨어하우스(Data Warehouse)가 특정 부서의 요구사항에 맞춰 구축되면서 발생했습니다. 서로 다른 기술 스택, 데이터 형식, 접근 권한 등으로 인해 데이터가 고립됩니다. **문제점:**

- **정보 단절:** 고객 데이터가 여러 곳에 흩어져 있어 전체적인 고객 뷰(Customer 360)를 얻기 어렵습니다.
- **비효율성:** 데이터 공유 및 접근이 어려워 비즈니스 기회를 놓치고 의사 결정이 지연됩니다.
- **벤더 종속성:** 종종 독점적인 시스템에 묶여 유연성이 떨어집니다.

데이터 레이크 (Data Lake)

한 줄 핵심 요약: 정형, 반정형, 비정형 등 모든 종류의 데이터를 원본 형태로 저장하는 거대한 중앙 집중식 저장소입니다. **직관적 비유:** 모든 종류의 물건을 일단 쌓아두는 거대한 창고와 같습니다. 나중에 필요할 때 찾아 쓰기 위해 일단 보관합니다. **기술적 설명:** 하둡(Hadoop) 시대에 인기를 얻었으며, 저렴한 클라우드 스토리지(예: S3, ADLS)를 기반으로 합니다. '스키마 온 리드(Schema on Read)' 방식을 채택하여 데이터 수집(Ingestion)이 매우 유연하고 쉽습니다. **장점:**

- **유연성:** 모든 형식의 데이터를 저장할 수 있어 미래의 분석 요구사항에 대비할 수 있습니다.
- **저렴한 비용:** 대규모 데이터를 저렴하게 저장할 수 있습니다.
- **확장성:** 스토리지와 컴퓨팅이 분리되어 있어 독립적으로 확장 가능합니다.
- **비정형 데이터 지원:** 이미지, 비디오, 텍스트 등 비정형 데이터 처리에 강합니다.

문제점 (관리 부실 시):

- **데이터 스왐프 (Data Swamp)로 변질:** 거버넌스, 접근 패턴, 품질 관리가 제대로 이루어지지 않으면 데이터가 무질서하게 쌓여 쓸모없어집니다. 데이터가 너무 많아 무엇이 어디에 있는지, 어떤 품질인지 알 수 없게 됩니다.
- **소비의 어려움:** 스키마 온 리드 방식은 수집은 쉽지만, 데이터를 소비(Consumption)할 때마다 스키마를 파악해야 하므로 복잡하고 비효율적입니다.

데이터 웨어하우스 (Data Warehouse)

한 줄 핵심 요약: 정형 데이터를 특정 비즈니스 분석 목적에 맞게 구조화하고 정제하여 저장하는 시스템입니다. **직관적 비유:** 잘 정리된 백화점 진열대와 같습니다. 필요한 물건을 쉽게 찾을 수 있도록 분류하고 라벨링되어 있습니다. **기술적 설명:** 관계형 데이터베이스(RDBMS) 기반으로, '스키마 온 라이트(Schema on Write)' 방식을 사용합니다. 데이터를 저장하기 전에 엄격한 스키마를 정의해야 합니다. 주로 비즈니스 인텔리전스(BI) 및 보고서 작성에 사용됩니다. **장점:**

- **높은 성능:** 정형 데이터에 대한 복잡한 쿼리를 빠르게 처리합니다.
- **데이터 품질 및 일관성:** 엄격한 스키마와 ETL(Extract, Transform, Load) 프로세스를 통해 높은 데이터 품질을 보장합니다.
- **쉬운 사용성:** 비즈니스 사용자가 SQL을 통해 쉽게 데이터를 분석할 수 있습니다.

문제점:

- **비용:** 대규모 데이터 저장 및 처리에 비용이 많이 듭니다.
- **유연성 부족:** 정형 데이터만 지원하며, 스키마 변경이 어렵습니다. 비정형 데이터 처리에 부적합합니다.
- **확장성 제한:** 스토리지와 컴퓨팅이 긴밀하게 연결되어 있어 확장성이 제한적일 수 있습니다.

데이터 레이크하우스 (Data Lakehouse)

한 줄 핵심 요약: 데이터 레이크의 유연성과 확장성에 데이터 웨어하우스의 안정성, 성능, 거버넌스 기능을 결합한 새로운 데이터 아키텍처입니다. **직관적 비유:** 페라리(웨어하우스의 속도)와 트랙터 트레일러(레이크의 용량)의 장점을 모두 가진 만능 차량과 같습니다. **기술적 설명:** 델타 레이크(Delta Lake)와 같은 오픈 소스 스토리지 포맷을 기반으로 합니다. 데이터 레이크 위에 ACID 트랜잭션, 스키마 관리, 데이터 버전 관리, 인덱싱 등의 기능을 추가하여 데이터 웨어하우스의 신뢰성을 제공합니다. **장점:**

- **모든 데이터 유형 지원:** 정형, 반정형, 비정형 데이터를 모두 처리합니다.
- **ACID 트랜잭션:** 데이터 무결성과 신뢰성을 보장합니다.
- **스키마 진화:** 스키마 변경에 유연하게 대응합니다.
- **성능:** 최적화된 쿼리 엔진(예: Photon)과 데이터 레이아웃(Z-ordering)을 통해 높은 분석 성능을 제공합니다.
- **통합 거버넌스:** 유니티 카탈로그(Unity Catalog)를 통해 데이터 접근 제어, 감사, 계보(Lineage) 관리가 용이합니다.
- **비용 효율성:** 저렴한 클라우드 스토리지를 활용하여 대규모 데이터를 효율적으로 관리합니다.
- **단일 플랫폼:** 데이터 엔지니어링, BI, AI/ML 워크로드를 단일 플랫폼에서 지원하여 복잡성을 줄입니다.

2. 델타 레이크 (Delta Lake): 레이크하우스의 기반 프로토콜

델타 레이크는 데이터 레이크의 유연성을 유지하면서 데이터 웨어하우스의 신뢰성을 제공하기 위해 개발된 오픈 소스 스토리지 레이어입니다. 파케이(Parquet) 파일 포맷 위에 구축되어, 데이터 레이크에 관계형 데이터베이스와 유사한 기능을 추가합니다.

Table 1: 데이터 웨어하우스 vs. 데이터 레이크 비교

특성	데이터 웨어하우스 (Data Warehouse)	데이터 레이크 (Data Lake)
스키마	스키마 온 라이트 (Schema on Write): 저장 전 엄격한 스키마 정의	스키마 온 리드 (Schema on Read): 저장 시 스키마 자유, 읽을 때 정의
데이터 유형	정형 데이터만 지원	정형, 반정형, 비정형 데이터 모두 지원
데이터 볼륨	제한적	대규모 (페타바이트 스케일 이상)
처리 방식	주로 배치 (Batch) 처리	배치 및 스트리밍 (Streaming) 처리 모두 지원
데이터 포맷	주로 독점적인 포맷	오픈 포맷 (Parquet, ORC, JSON 등)
비용	데이터 볼륨에 비례하여 비용 증가 (컴퓨팅과 스토리지 결합)	스토리지와 컴퓨팅 분리, 대규모 데이터 저장 비용 효율적
성능	정형 데이터에 대한 쿼리 응답 속도 매우 빠름	유연하나, 특정 쿼리에서 웨어하우스보다 느릴 수 있음 (레이크하우스에서 개선)
주요 사용자	비즈니스 사용자, BI 분석가	데이터 과학자, 데이터 엔지니어, ML 엔지니어
주요 용도	비즈니스 보고서, 분석, BI	AI/ML 모델링, 탐색적 분석, 원시 데이터 저장

델타 레이크의 주요 기능

- **ACID 트랜잭션 (Atomicity, Consistency, Isolation, Durability):**

- **왜 필요한가?:** 분산 컴퓨팅 환경에서 데이터 변경 시 부분적인 실패나 동시성 문제로 데이터가 손상되거나 일관성이 깨질 수 있습니다. 델타 레이크는 이러한 문제를 해결하여 데이터 무결성을 보장합니다.
- **설명:** 여러 작업이 동시에 데이터를 읽고 쓸 때, 데이터가 항상 일관된 상태를 유지하도록 합니다. 예를 들어, 데이터 업데이트 중 오류가 발생하면 전체 작업이 롤백되어 이전 상태로 돌아갑니다(원자성). 사용자는 항상 완전하고 일관된 데이터를 보게 됩니다(고립성).

- **스키마 진화 (Schema Evolution):**

- **왜 필요한가?:** 데이터 소스의 스키마는 시간이 지남에 따라 변경될 수 있습니다(예: 새 컬럼 추가). 엄격한 스키마 검사는 이러한 변화에 유연하게 대응하기 어렵게 만듭니다.
- **설명:** 데이터 스키마가 변경될 때(예: 새 컬럼 추가, 데이터 타입 변경) 기존 파이프라인을 중단하지 않고도 데이터를 계속 쓸 수 있도록 지원합니다. `mergeSchema=true` 옵션을 통해 자동으로 스키마를 병합할 수 있습니다.

- **타임 트래블 (Time Travel):**

- **왜 필요한가?:** 데이터 오류 발생 시 과거 시점으로 데이터를 복구하거나, 특정 시점의 데이터를 분석해야 할 필요가 있습니다.
- **설명:** 델타 레이크는 모든 트랜잭션 변경 이력을 저장하는 트랜잭션 로그를 유지합니다. 이를 통해 특정 버전 번호나 타임스탬프를 사용하여 과거 데이터 상태를 조회하거나 복구할 수 있습니다. 이는 데이터 감사, 재현 가능한 실험, 오류 복구에 매우 유용합니다.

- **데이터 스키핑 (Data Skipping) 및 Z-오더링 (Z-ordering):**

- **왜 필요한가?:** 대규모 데이터에서 쿼리 성능을 최적화하려면 불필요한 데이터 읽기를 최소화해야 합니다.

- **설명:** 델타 레이크는 파일 레벨의 통계(최소/최대 값 등)를 메타데이터로 저장하여 쿼리 시 관련 없는 데이터 파일을 건너뛸 수 있습니다(데이터 스키핑). Z-오더링은 여러 컬럼에 대한 쿼리 성능을 향상시키기 위해 데이터를 물리적으로 재구성하는 기술로, 데이터 지역성을

3. 스파크 (Spark) 핵심 개념 및 성능 최적화

스파크는 대규모 데이터 처리를 위한 강력한 분산 컴퓨팅 프레임워크입니다. 스파크의 효율적인 동작을 이해하고 성능 병목 현상을 해결하는 것은 데이터 엔지니어에게 필수적입니다.

DAG (Directed Acyclic Graph)

한 줄 핵심 요약: 스파크에서 데이터 변환 및 액션의 논리적인 실행 계획을 나타내는 방향성 있는 비순환 그래프입니다. **직관적 비유:** 요리 레시피와 같습니다. 재료 준비(데이터 로드)부터 최종 요리(액션)까지의 모든 단계와 순서가 명확하게 정의되어 있습니다. 중간에 순환(Cycle)이 없어 무한 루프에 빠지지 않습니다. **기술적 설명:** 스파크는 '지연 평가(Lazy Evaluation)' 방식을 사용합니다. 즉, `map`, `filter`와 같은 변환(Transformation) 작업은 즉시 실행되지 않고 DAG에 추가됩니다. `collect`, `count`와 같은 액션(Action)이 호출될 때 비로소 DAG가 최적화되고 물리적 실행 계획으로 변환되어 실행됩니다. **구성 요소:**

- **노드 (Nodes):** 데이터 변환(Transformation) 또는 액션(Action)을 나타냅니다.
- **엣지 (Edges):** 데이터 흐름의 방향과 의존성을 나타냅니다.

스파크 잡(Job)의 구성

한 줄 핵심 요약: 스파크 잡은 여러 스테이지(Stage)로 구성되며, 각 스테이지는 여러 태스크(Task)로 이루어집니다. **직관적 비유:** 큰 프로젝트(잡)를 여러 단계(스테이지)로 나누고, 각 단계는 여러 작은 업무(태스크)로 구성하는 것과 같습니다. **기술적 설명:**

- **스파크 잡 (Spark Job):** 하나의 액션(예: `count()`, `collect()`)이 트리거될 때 생성되는 최상위 작업 단위입니다.
- **스테이지 (Stage):** 셔플(Shuffle) 작업이 발생할 때마다 새로운 스테이지가 시작됩니다. 셔플은 데이터가 워커 노드 간에 교환되어야 하는 작업(예: `groupByKey`, `sort`)을 의미합니다. 셔플이 없는 변환(예: `map`, `filter`)은 동일한 스테이지 내에서 파이프라인으로 처리됩니다.
- **태스크 (Task):** 각 스테이지 내에서 워커 노드의 코어에서 실행되는 가장 작은 작업 단위입니다. 데이터 파티션 하나당 하나의 태스크가 할당됩니다.

4. 메달리온 아키텍처 (Medallion Architecture)

메달리온 아키텍처는 데이터 레이크하우스에서 데이터의 품질과 정제 수준에 따라 데이터를 계층화하는 모범 사례입니다. 데이터가 원본에서 최종 소비 단계로 이동함에 따라 점진적으로 정제되고 구조화됩니다.

Table 2: 스파크 성능 병목 현상 및 해결 전략

병목 현상	설명	해결 전략
스필 (Spill)	스파크 작업 시 할당된 메모리가 부족하여 중간 결과가 디스크에 기록되는 현상. 디스크 I/O는 메모리보다 훨씬 느려 성능 저하의 주범.	• 워커 노드에 더 많은 메모리 할당. • 데이터 처리 단위를 작게 나누어 메모리 사용량 줄이기. • 작업 특성 분석 후 메모리 요구량 예측.
셔플 (Shuffle)	분산된 워커 노드 간에 데이터를 교환해야 하는 작업 (예: <code>groupBy</code> , <code>join</code> , <code>sort</code>). 네트워크 I/O가 발생하여 비용이 많이 들고 느림.	• 셔플을 유발하는 작업을 최소화하도록 코드 최적화. • 데이터 파티셔닝 전략을 잘 설계하여 셔플 필요성 줄이기. • 브로드캐스트 조인 (Broadcast Join) 활용 (작은 테이블 조인 시).
데이터 스큐 (Data Skew)	특정 파티션에 데이터가 불균형하게 분포되어 일부 워커 노드에 작업 부하가 집중되는 현상. 이로 인해 해당 노드가 '스트래글러 (Straggler)'가 되어 전체 작업 완료 시간을 지연시킴.	• 스큐가 발생하지 않도록 파티셔닝 키 재설계 (예: 고유성이 높은 컬럼 사용). • 스큐된 키에 대한 데이터를 미리 필터링하거나 샘플링. • 스큐된 키를 분할하여 여러 태스크에 분산 (Salting 기법).
작은 파일 문제 (Small Files Problem)	너무 많은 수의 작은 파일들이 클라우드 스토리지에 저장되어 메타데이터 관리 및 I/O 오버헤드가 커지는 문제.	• 파일 압축 (Compaction) 작업을 통해 작은 파일들을 큰 파일로 병합 (OPTIMIZE 명령). • 델타 레이크의 OPTIMIZE 및 ZORDER 기능을 활용하여 파일 레이아웃 최적화.

메달리온 아키텍처의 3단계

- **브론즈 레이어 (Bronze Layer):**
 - **설명:** 원본 시스템에서 들어오는 데이터를 거의 변경 없이 그대로 저장하는 영역입니다. 원시 데이터(Raw Data)의 불변 기록(Immutable Record) 역할을 합니다.
 - **특징:** 스키마 온 리드, 모든 데이터 형식 지원, 히스토리컬 데이터 보관, 재처리(Reprocessing) 가능.
 - **예시:** CSV, JSON, Parquet 등 다양한 형식의 원본 로그 파일, 트랜잭션 데이터.
- **실버 레이어 (Silver Layer):**
 - **설명:** 브론즈 레이어의 데이터를 정제, 필터링, 보강(Augment)하여 신뢰할 수 있는(Trusted) 데이터를 생성하는 영역입니다. 조인(Join) 및 조회(Lookup) 작업이 주로 이루어집니다.
 - **특징:** 스키마 온 라이트 또는 스키마 진화, 데이터 품질 향상, 중복 제거, 결측치 처리, 데이터 표준화.
 - **예시:** 고객 정보와 주문 정보를 조인하여 정제된 고객 주문 데이터셋.
- **골드 레이어 (Gold Layer):**
 - **설명:** 실버 레이어의 데이터를 특정 비즈니스 사용 사례(BI, AI/ML)에 맞게 집계, 요약, 변환하여 저장하는 영역입니다. 여러 골드 레이어가 존재할 수 있습니다.
 - **특징:** 스키마 온 라이트, 높은 성능, 비즈니스 의미론(Semantics) 적용, KPI 정의.
 - **예시:** 월별 판매 보고서, 특정 제품군에 대한 ML 모델 학습용 피처(Feature) 테이블.

5. 데이터 수집 (Data Ingestion) 도구

데이터 레이크하우스로 데이터를 가져오는 방법은 다양하며, Databricks는 여러 효율적인 도구를 제공합니다.

Databricks 데이터 수집 도구

- **오토로더 (Autoloader):**
 - **설명:** 클라우드 스토리지(S3, ADLS, GCS)에서 새로운 파일이 도착하는 것을 지속적으로 감지하여 델타 테이블로 증분 로드(Incrementally Ingest) 하는 도구입니다.
 - **특징:** 매우 확장 가능(수십억 파일), 스키마 추론 및 진화 지원, 스트리밍 친화적, cloudFiles 포맷 사용.
 - **용도:** 실시간 또는 준실시간으로 지속적으로 발생하는 파일 데이터 수집.
- **카피 인투 (COPY INTO):**
 - **설명:** 클라우드 스토리지에서 델타 테이블로 대량의 데이터를 한 번에 로드하는 SQL 명령입니다.
 - **특징:** 멍등성(Idempotent) 보장 (동일 파일을 여러 번 실행해도 중복 방지), 간단한 SQL 구문, 대규모 배치(Batch) 수집에 적합.
 - **용도:** 초기 대량 데이터 로드, 특정 시점의 배치 데이터 수집.
- **로컬 파일 업로드 (Local File Upload):**
 - **설명:** Databricks UI를 통해 로컬 컴퓨터의 파일을 직접 업로드하여 테이블을 생성하는 기능입니다.
 - **특징:** 최대 10개 파일, 총 2GB 제한, CSV, JSON, Parquet 등 다양한 포맷 지원.
 - **용도:** 비즈니스 사용자가 소규모 데이터를 빠르게 분석하고자 할 때 유용.
- **델타 라이브 테이블 (Delta Live Tables, DLT):**
 - **설명:** 데이터 파이프라인을 선언적으로 정의하고 관리하는 프레임워크입니다. 데이터 품질 제약 조건, 자동 복구, 모니터링 기능이 내장되어 있습니다.
 - **특징:** 데이터 품질(Quality)에 중점, 자동화된 인프라 관리, 스파크 스트럭처드 스트리밍(Spark Structured Streaming) 기반.
 - **용도:** 복잡한 ETL/ELT 파이프라인 구축 및 운영.

6. 데이터 거버넌스 (Data Governance)

데이터 거버넌스는 데이터의 가용성, 사용성, 무결성, 보안을 보장하기 위한 정책, 프로세스, 역할 및 책임의 집합입니다. 레이크하우스 아키텍처에서 거버넌스는 데이터의 신뢰성을 확보하고 가치를 창출하는데 필수적입니다.

데이터 거버넌스의 중요성

- **데이터 발견 가능성 (Discoverability):** 사용자가 필요한 데이터를 쉽게 찾을 수 있도록 메타데이터(Metadata)를 풍부하게 관리하고 카탈로그화해야 합니다.
- **접근 제어 (Access Control):** 모든 데이터가 모든 사람에게 공개되어서는 안 됩니다. 민감한 데이터(예: 개인 식별 정보, 금융 정보)에 대한 행(Row) 수준 또는 컬럼(Column) 수준의 세분화된 접근 권한 관리가 필요합니다.
- **데이터 품질 (Data Quality):** 데이터의 정확성, 완전성, 일관성, 최신성을 보장해야 합니다. DLT와 같은 도구는 파이프라인 내에서 품질 제약 조건을 정의하고 적용할 수 있습니다.
- **데이터 계보 (Data Lineage):** 데이터가 원본에서 최종 소비 단계까지 어떻게 변환되었는지 추적할 수 있어야 합니다. 이는 데이터의 신뢰성을 평가하고 문제 발생 시 원인을 파악하는 데 중요합니다.
- **감사 (Audit):** 누가 언제 어떤 데이터에 접근했는지, 어떤 변경을 가했는지 기록하고 감사할 수 있어야 합니다.

Databricks의 유니티 카탈로그 (Unity Catalog): 데이터 및 AI 자산에 대한 통합 거버넌스 솔루션으로, 메타데이터 관리, 세분화된 접근 제어, 데이터 계보 추적 등을 지원하여 데이터 거버넌스를 중앙에서 효율적으로 관리할 수 있도록 돕습니다.

7. Databricks 작업 (Jobs) 및 워크플로우 오케스트레이션

Databricks Jobs는 데이터 파이프라인을 구성하는 여러 태스크(Task)를 정의하고, 이들의 실행 순서와 의존성을 관리하며, 스케줄링, 모니터링, 오류 처리 등을 자동화하는 기능입니다. 이는 DAG(Directed Acyclic Graph) 개념을 기반으로 합니다.

Databricks Jobs의 주요 기능

- **태스크 (Tasks) 및 의존성:**
 - **설명:** 각 잡은 여러 태스크로 구성될 수 있으며, 각 태스크는 노트북, 파이썬 스크립트, SQL 쿼리, JAR 파일 등 다양한 형태로 정의될 수 있습니다. 태스크 간에는 의존성을 설정하여 특정 태스크가 완료된 후에 다음 태스크가 실행되도록 할 수 있습니다.
 - **예시:** 클릭 데이터 수집 태스크와 주문 데이터 수집 태스크는 독립적으로 실행될 수 있지만, 사용자 세션화 태스크는 클릭 데이터 수집 태스크가 완료된 후에만 실행될 수 있습니다.
- **스케줄링 (Scheduling):**
 - **설명:** 크론 표현식(Cron Expression)을 사용하여 특정 시간/주기로 잡을 실행하거나, 파일 도착(File Arrival)과 같은 이벤트 트리거, 또는 지속적인(Continuous) 실행 모드를 설정할 수 있습니다.
- **오류 처리 및 재시도 (Retries):**
 - **설명:** 일시적인 오류(예: 네트워크 문제) 발생 시 자동으로 작업을 재시도하도록 설정할 수 있습니다. 타임아웃(Timeout) 설정도 가능합니다.
- **알림 (Alerts):**
 - **설명:** 작업 실패, 성공, 타임아웃 등 특정 이벤트 발생 시 파이프라인 소유자에게 알림을 보낼 수 있습니다.
- **수리 후 실행 (Repair and Run):**
 - **설명:** 여러 태스크로 구성된 잡에서 특정 태스크가 실패했을 때, 실패한 태스크부터 다시 실행하여 시간과 컴퓨팅 비용을 절약할 수 있습니다.
- **파라미터 전달 (Parameter Passing):**
 - **설명:** 잡 수준 파라미터(Job Parameters)와 태스크 수준 파라미터(Task Parameters)를 정의하여 환경 설정이나 동적인 값들을 파이프라인에 전달할 수 있습니다.

4 데이터 아키텍처 패러다임: 데이터 메시 vs. 데이터 패브릭

데이터 관리의 복잡성이 증가함에 따라, 조직은 데이터를 더 효율적으로 관리하고 활용하기 위한 새로운 아키텍처 패러다임을 모색하고 있습니다. 데이터 메시와 데이터 패브릭은 이러한 노력의 대표적인 예시입니다.

Table 3: 데이터 메시 vs. 데이터 패브릭 비교

특성	데이터 메시 (Data Mesh)	데이터 패브릭 (Data Fabric)
핵심 아이디어	분산된 도메인 중심의 데이터 소유권: 데이터를 제품처럼 취급하고 도메인 팀이 소유 및 관리.	기술 중심의 통합 아키텍처: 다양한 데이터 소스를 통합하여 단일 환경처럼 제공.
초점	조직 운영 모델 및 문화적 변화, 책임 분산.	기술 및 자동화, 데이터 흐름 및 연결.
데이터 소유권	각 도메인 팀이 데이터 제품을 소유, 큐레이션, 서비스.	중앙 팀이 통합 및 거버넌스 패브릭 관리 (일부 위임).
거버넌스	공통 표준을 가진 연합 컴퓨팅 거버넌스 (Federated Computational Governance).	메타데이터 카탈로그 및 자동화를 통한 중앙 집중식 거버넌스.
데이터 접근	데이터 제품은 발견 가능하고, 주소 지정 가능하며, 신뢰할 수 있어야 함.	원활한 데이터 접근 및 통합 경험 제공.
확장성	조직 복잡성에 따라 확장 (도메인 팀 추가).	기술 복잡성에 따라 확장 (하이브리드, 멀티클라우드, 레거시 통합).
주요 사용자	중앙 집중식 데이터 팀이 병목인 대규모 기업.	여러 환경(온프레미스, 클라우드, SaaS)에 데이터가 분산된 기업.
비유	각 도시가 공통 규칙 하에 자체 인프라를 관리하는 연합.	모든 도시를 가로지르는 고속 철도 및 고속도로 시스템.

데이터 메시의 4가지 핵심 원칙

- **도메인 소유권 (Domain Ownership):**
 - **설명:** 데이터를 생성하고 가장 잘 이해하는 비즈니스 도메인 팀이 데이터 자산과 파이프라인에 대한 책임을 집니다. 이는 중앙 IT 팀의 병목 현상을 해소하고 민첩성을 높입니다.
 - **Databricks 지원:** 오픈 및 유연한 아키텍처, 데이터 자산 및 파이프라인의 분산 소유권.
- **데이터를 제품으로 (Data as a Product):**
 - **설명:** 데이터는 단순히 부산물이 아니라, 고객(다른 도메인 팀이나 분석가)에게 가치를 제공하는 제품으로 취급되어야 합니다. 이는 데이터 계약, 품질, 발견 가능성, 신뢰성을 강조합니다.
 - **Databricks 지원:** 오픈 표준, 오픈 포맷, ACID 보장, 버전 관리, 감사, DLT를 통한 신선하고 고품질 데이터.
- **셀프 서비스 데이터 플랫폼 (Self-Service Data Platform):**
 - **설명:** 도메인 팀이 중앙 IT의 개입 없이 데이터를 생성, 관리, 소비할 수 있도록 필요한 도구와 인프라를 제공합니다.
 - **Databricks 지원:** 통합 플랫폼, 관리형 오케스트레이션, 자동 스케일링, 서버리스 인프라, Infrastructure as Code.
- **연합 컴퓨팅 거버넌스 (Federated Computational Governance):**
 - **설명:** 중앙에서 정의된 공통 표준과 규칙(예: 보안, 개인 정보 보호)을 따르면서도, 각 도메인 팀이 자신의 데이터에 대한 세부적인 거버넌스 결정을 내릴 수 있도록 합니다.
 - **Databricks 지원:** 유니티 카탈로그를 통한 데이터 발견, 접근 제어, 계보 관리.

5 절차 및 방법: Databricks Jobs 워크플로우 오케스트레이션

Databricks Jobs는 복잡한 데이터 파이프라인을 시각적으로 정의하고 관리할 수 있는 강력한 도구입니다. 다음은 잡을 생성하고 관리하는 주요 단계와 기능입니다.

1. 잡 생성 및 태스크 정의

1. Databricks 워크스페이스에서 'Jobs' 또는 'Pipelines' 섹션으로 이동합니다.
2. 'Create Job'을 클릭하여 새로운 잡을 생성합니다.
3. **태스크 추가:** 잡 내에 하나 이상의 태스크를 추가합니다. 각 태스크는 다음 중 하나일 수 있습니다.
 - **노트북 (Notebook):** Python, Scala, SQL, R 코드를 포함하는 Databricks 노트북.
 - **파이썬 스크립트 (Python Script):** Python 파일.
 - **SQL 쿼리 (SQL Query):** SQL 파일 또는 직접 입력하는 SQL 쿼리.
 - **JAR 파일 (JAR File):** 컴파일된 Java/Scala 코드.
 - **파이썬 휠 (Python Wheel):** Python 패키지.
4. **태스크 이름 지정:** 각 태스크에 고유한 이름을 부여합니다.
5. **코드 경로 지정:** 노트북 또는 스크립트의 경우 워크스페이스 내 경로 또는 Git 저장소 경로를 지정합니다.
6. **컴퓨팅 설정:** 태스크를 실행할 컴퓨팅 자원(클러스터)을 선택합니다. 서버리스(Serverless) 컴퓨팅을 사용하면 인프라 관리가 간소화됩니다.

2. 태스크 의존성 설정

1. 태스크를 추가할 때 'Depends on' 옵션을 사용하여 다른 태스크에 대한 의존성을 설정할 수 있습니다.
2. 의존성이 없는 태스크는 병렬로 실행될 수 있습니다.
3. **예시:** '클릭 데이터 수집'과 '주문 데이터 수집'은 병렬로 실행하고, 이 두 태스크가 완료된 후에 '사용자 세션화' 태스크가 실행되도록 설정할 수 있습니다.

3. 파라미터 전달

1. **잡 파라미터 (Job Parameters):** 워크플로우 전체에 적용되는 파라미터입니다. 환경(예: dev/prod) 설정 등에 사용됩니다.
2. **태스크 파라미터 (Task Parameters):** 특정 태스크에만 전달되는 파라미터입니다.
3. **사용 방법 (Python 노트북 예시):**

```

1 # 잡파라미터가져오기
2 job_param_value = dbutils.widgets.get("job_param_name")
3 print(f"Job Parameter: {job_param_value}")
4
5 # 태스크파라미터가져오기
6 task_param_value = dbutils.widgets.get("task_param_name")
7 print(f"Task Parameter: {task_param_value}")

```

Listing 1: Databricks 노트북에서 파라미터 사용 예시

4. 스케줄링 및 트리거

1. **스케줄링 (Scheduled):** 크론 표현식을 사용하여 매일, 매주, 특정 요일 등 정해진 시간에 잡을 실행합니다.
2. **파일 도착 (File Arrival):** 특정 클라우드 스토리지 경로에 새로운 파일이 도착할 때 잡을 트리거합니다.
3. **연속 (Continuous):** 잡이 항상 실행 상태를 유지하며, 새로운 데이터가 도착하는 즉시 처리합니다. (컴퓨팅 비용이 높음)

5. 모니터링 및 오류 처리

1. **실행 기록 (Run History):** 각 잡의 실행 기록(시작 시간, 기간, 상태, 실행 ID)을 확인할 수 있습니다.
2. **알림 (Notifications):** 잡 실패, 성공, 타임아웃 등 특정 이벤트 발생 시 이메일 또는 다른 채널로 알림을 설정할 수 있습니다.
3. **재시도 (Retries):** 일시적인 오류 발생 시 자동으로 잡을 재시도하도록 설정할 수 있습니다.
4. **수리 후 실행 (Repair and Run):** 실패한 잡의 경우, 실패 지점부터 다시 실행하여 이미 성공한 태스크는 건너뛰고 시간과 비용을 절약합니다.

6 실습: 델타 레이크 기능 활용

이 섹션에서는 델타 레이크의 핵심 기능인 ACID 트랜잭션, 타임 트래블, 데이터 조작(INSERT, DELETE, UPDATE, MERGE)을 실습 예시를 통해 살펴봅니다.

1. 델타 테이블 생성 및 기본 조회

먼저, 기존 파케이(Parquet) 데이터를 사용하여 델타 테이블을 생성하고 기본 데이터를 확인합니다.

```

1  -- 컴퓨팅에 연결
2  -- (Databricks 에서 UI 클러스터 또는 SQL 웨어하우스에 연결 )
3
4  -- 카탈로그 및 스키마 설정
5  USE CATALOG <your_catalog>;
6  CREATE SCHEMA IF NOT EXISTS <your_schema>;
7  USE SCHEMA <your_schema>;
8
9  -- 기존 테이블이 있다면 삭제 ( 편의를 위해 )
10 DROP TABLE IF EXISTS loans_by_state_delta;
11
12 -- Parquet 데이터를 기반으로 델타 테이블 생성
13 -- Databricks 샘플 데이터셋 사용 (lending_club)
14 CREATE TABLE loans_by_state_delta
15 USING DELTA
16 AS SELECT state, COUNT(*) AS loan_count
17 FROM parquet.`texttt{/databricks-datasets/samples/lending_club/parquet/}`
18 GROUP BY state;
19
20 -- 생성된 델타 테이블의 상세 정보 확인
21 DESCRIBE DETAIL loans_by_state_delta;
22
23 -- 데이터 조회 ( : 캘리포니아, 뉴욕 등 )
24 SELECT * FROM loans_by_state_delta ORDER BY state;

```

Listing 2: 델타 테이블 생성 및 데이터 조회

설명:

- DESCRIBE DETAIL 명령은 델타 테이블의 포맷, ID, 위치, 생성 시간, 속성 등 상세한 메타데이터를 보여줍니다.
- 초기 데이터에는 'Iowa' 주(State)의 데이터가 포함되어 있지 않음을 확인할 수 있습니다.

2. 델타 테이블 스트리밍 읽기 및 배치 쓰기

델타 테이블은 스트리밍 소스로도 활용될 수 있으며, 동시에 배치 쓰기 작업도 지원합니다.

```

1  -- 델타 테이블을 스트리밍 소스로 읽기 (Python 예시)
2  -- 이 스트림은 백그라운드에서 실행되며 , 새로운 변경 사항을 감지합니다 .
3  -- 실제 환경에서는 별도의 노트북 잡에서 / 실행하는 것이 일반적입니다 .
4  -- 여기서는 개념 설명을 위해 잠시 실행 후 중지합니다 .

```

```

5 -- (Databricks 노트북에서 Python 셀로실행 )
6 -- spark.readStream.format("delta").table("loans_by_state_delta").writeStream.
  format("console").start()
7
8 -- Iowa 데이터배치삽입회 (6 반복)
9 -- 이작업은스트림이실행중인동안에도안전하게수행됩니다
10 -- (SQL 셀로실행 )
11 INSERT INTO loans_by_state_delta VALUES ('IA', 450);
12 -- 이 INSERT 문을 6 번실행하여의 Iowa 를 loan_count 으로 2700 만듭니다.

```

Listing 3: 델타 테이블 스트리밍 읽기 및 배치 쓰기

설명:

- 델타 테이블은 `spark.readStream.format("delta").table(...)`을 사용하여 스트리밍 소스로 읽을 수 있습니다. 이는 새로운 데이터가 테이블에 추가될 때마다 자동으로 감지하여 처리합니다.
- ACID 트랜잭션 덕분에, 스트리밍 읽기 작업이 진행 중인 동안에도 INSERT와 같은 배치 쓰기 작업이 안전하게 수행됩니다. 읽기 작업은 쓰기 작업이 완료되기 전까지는 이전 버전의 데이터를 보게 됩니다.
- Iowa 데이터 삽입 후, 테이블을 다시 조회하면 Iowa의 `loan_count` .

3. 세분화된 데이터 조작 (DELETE, UPDATE, MERGE)

델타 레이크는 관계형 데이터베이스처럼 특정 행(Row)을 대상으로 하는 DELETE, UPDATE, MERGE 작업을 지원합니다. 이는 기존 파케이(Parquet) 테이블에서는 불가능했던 기능입니다.

```

1 -- Iowa 데이터삭제
2 -- 이전 ( 단계에서삽입된 Iowa 데이터를삭제합니다 .)
3 DELETE FROM loans_by_state_delta WHERE state = 'IA';
4
5 -- 삭제후데이터확인
6 SELECT * FROM loans_by_state_delta ORDER BY state;
7
8 -- 데이터업데이트예 (: 캘리포니아의를 loan_count 으로 2500 업데이트)
9 UPDATE loans_by_state_delta SET loan_count = 2500 WHERE state = 'CA';
10
11 -- 업데이트후데이터확인
12 SELECT * FROM loans_by_state_delta ORDER BY state;
13
14 -- MERGE INTO (Upsert) 작업
15 -- 새로운데이터변경 ( 레코드)를 시뮬레이션하는임시뷰생성
16 CREATE OR REPLACE TEMPORARY VIEW new_loan_data AS
17 SELECT 'IA' AS state, 10 AS loan_count
18 UNION ALL SELECT 'CA' AS state, 2500 AS loan_count
19 UNION ALL SELECT 'OR' AS state, 500 AS loan_count; -- 은Oregon 기존테이블에없음
20
21 -- MERGE INTO 실행
22 MERGE INTO loans_by_state_delta AS target

```

```

23 USING new_loan_data AS source
24 ON target.state = source.state
25 WHEN MATCHED THEN
26     UPDATE SET target.loan_count = source.loan_count
27 WHEN NOT MATCHED THEN
28     INSERT (state, loan_count) VALUES (source.state, source.loan_count);
29
30 -- MERGE 후데이터확인
31 SELECT * FROM loans_by_state_delta ORDER BY state;

```

Listing 4: 델타 테이블 데이터 조작 (DELETE, UPDATE, MERGE)

설명:

- DELETE 및 UPDATE 명령은 특정 조건에 맞는 행을 직접 삭제하거나 수정합니다. 기존 파케이 테이블에서는 전체 파티션을 다시 작성해야 했지만, 델타 레이크는 트랜잭션 로그를 통해 효율적으로 처리합니다.
- MERGE INTO는 '업서트(Upsert)' 기능을 제공합니다. 소스 데이터와 타겟 테이블을 비교하여, 일치하는 레코드는 업데이트하고, 일치하지 않는 레코드는 삽입합니다. 이 모든 작업이 단일 원자적(Atomic) 트랜잭션으로 처리됩니다.
- 위 예시에서는 'IA'와 'CA'는 업데이트되고, 'OR'은 새로 삽입됩니다.

4. 타임 트래블 (Time Travel)

델타 레이크는 모든 변경 이력을 버전으로 관리하므로, 과거 특정 시점의 데이터 상태를 조회할 수 있습니다.

```

1 -- 델타테이블의변경이력확인
2 DESCRIBE HISTORY loans_by_state_delta;
3
4 -- 특정버전의데이터조회예      (: 버전 0)
5 -- 버전은 0 테이블이처음생성되었을때의상태입니다      .
6 -- 이때는 Iowa 데이터가없었을것입니다      .
7 SELECT * FROM loans_by_state_delta VERSION AS OF 0 ORDER BY state;
8
9 -- 특정버전의데이터조회예      (: MERGE 작업후의최신버전      )
10 -- DESCRIBE HISTORY 결과에서 MERGE 작업에해당하는버전번호를확인하여사용합니다      .
11 -- 예를들어 , 가MERGE 버전였다면 9:
12 SELECT * FROM loans_by_state_delta VERSION AS OF 9 ORDER BY state;
13
14 -- 특정타임스탬프시점의데이터조회예      (: '2023-01-01 10:00:00' 이전데이터 )
15 -- 실제타임스탬프는 DESCRIBE HISTORY 결과에서확인하여사용합니다      .
16 -- SELECT * FROM loans_by_state_delta TIMESTAMP AS OF '2023-01-01 10:00:00 '
    ORDER BY state;

```

Listing 5: 델타 테이블 타임 트래블

설명:

- DESCRIBE HISTORY 명령은 델타 테이블에 대한 모든 트랜잭션(CREATE, INSERT, DELETE, UPDATE, MERGE 등)의 기록을 보여줍니다. 각 트랜잭션에는 고유한 버전 번호와 타임스탬프가 부여됩니다.
- `VERSION AS OF <versionnumber> TIMESTAMP AS OF '<timestamp>'` .
- 이를 통해 데이터 오류 발생 시 쉽게 과거 상태로 돌아가 데이터를 복구하거나, 특정 시점의 데이터를 기반으로 분석을 재현할 수 있습니다.

7 FAQ (자주 묻는 질문)

초심자를 위한 FAQ

- **Q: 데이터 레이크와 데이터 스웜프는 어떻게 다른가요? A:** 데이터 레이크는 모든 데이터를 저장하는 '잠재력 있는' 저장소입니다. 반면 데이터 스웜프는 관리(거버넌스, 메타데이터, 품질)가 부실하여 데이터가 무질서하게 쌓여 쓸모없어진 데이터 레이크를 의미합니다. 레이크가 스웜프가 되지 않도록 잘 관리하는 것이 중요합니다.
- **Q: 스키마 온 리드와 스키마 온 라이트는 무엇인가요? A:** 스키마 온 라이트는 데이터를 저장하기 전에 엄격한 스키마(데이터 구조)를 정의해야 하는 방식입니다(예: 관계형 데이터베이스). 스키마 온 리드는 데이터를 저장할 때는 스키마에 구애받지 않고, 데이터를 읽을 때 스키마를 파악하는 방식입니다(예: 데이터 레이크). 스키마 온 리드는 유연하지만 데이터 소비를 어렵게 만들 수 있습니다.
- **Q: 스파크에서 '스필(Spill)'이 발생하면 어떻게 해야 하나요? A:** 스필은 메모리 부족으로 인해 중간 데이터가 디스크에 기록되는 현상으로, 성능 저하의 주요 원인입니다. 이를 피하려면 워커 노드에 더 많은 메모리를 할당하거나, 데이터 처리 로직을 최적화하여 메모리 사용량을 줄여야 합니다.
- **Q: 델타 레이크가 레이크하우스의 필수 구성 요소인가요? A:** 델타 레이크는 레이크하우스를 구축하는 '하나의 강력한 옵션'입니다. 델타 레이크 외에도 아이스버그(Iceberg), 후디(Hudi)와 같은 다른 오픈 소스 스토리지 포맷들도 레이크하우스 기능을 제공합니다. 하지만 델타 레이크는 ACID 트랜잭션, 스키마 진화, 타임 트래블 등 핵심 기능을 제공하여 레이크하우스 패러다임을 주도하고 있습니다.
- **Q: 빅데이터 시스템에서 관계형 데이터베이스처럼 인덱스를 만들 수 있나요? A:** 관계형 데이터베이스의 'CREATE INDEX'와 같은 직접적인 인덱스 개념은 빅데이터 시스템에 존재하지 않습니다. 대신 파티셔닝(Partitioning), 클러스터링(Clustering), Z-오더링(Z-ordering)과 같은 데이터 레이아웃 최적화 기법을 사용하여 쿼리 성능을 향상시킵니다. 이는 데이터의 물리적 저장 방식을 최적화하여 불필요한 데이터 스캔을 줄이는 방식입니다.
- **Q: Databricks는 실시간(Real-time) 데이터 처리를 지원하나요? DynamoDB처럼 밀리초 단위의 응답 속도가 가능한가요? A:** Databricks는 스파크 스트럭처드 스트리밍을 통해 실시간에 가까운(Near Real-time) 데이터 처리를 지원합니다. 수백 밀리초 단위의 쿼리 응답 속도는 가능하지만, DynamoDB나 Cassandra와 같은 운영 데이터베이스(Operational Database)의 단일 밀리초 응답 속도와는 목적이 다릅니다. Databricks는 주로 분석 워크로드에 최적화되어 있습니다. 최근에는 'Lakebase'와 같은 시도를 통해 운영 데이터베이스 기능도 탐색하고 있습니다.
- **Q: Databricks를 운영 데이터 소스(ODS)로 사용할 수 있나요? A:** 전통적으로 Databricks는 분석 시스템으로 설계되었기 때문에 운영 데이터 소스(ODS)로 직접 사용하기에는 적합하지 않았습니다. 하지만 'Lakebase'와 같은 새로운 이니셔티브를 통해 포스트그레스(PostgreSQL)와 같은 운영 데이터베이스 기능을 델타 테이블과 통합하려는 시도가 진행 중입니다. 아직 초기 단계이지만, 미래에는 ODS 역할도 가능할 수 있습니다.
- **Q: Databricks Jobs에서 병렬 태스크를 실행할 수 있나요? A:** 네, 가능합니다. 태스크 간에 의존성이 없는 경우, Databricks Jobs는 해당 태스크들을 병렬로 실행할 수 있습니다. 태스크 설정에서 'Depends on' 옵션을 제거하면 병렬 실행이 가능해집니다.

8 빠르게 훑어보기 (1페이지 요약)

데이터 아키텍처의 진화

데이터 사일로 → 데이터 레이크 (유연성, 저비용) → 데이터 웨어하우스 (성능, 신뢰성) → 레이크 하우스 (둘의 장점 결합)

데이터 레이크의 문제점

- 데이터 스웸프: 관리 부실로 인한 무질서.
- 소비의 어려움: 스키마 온 리드로 인한 복잡성.
- 신뢰성 부족: ACID 트랜잭션 부재.

델타 레이크: 레이크하우스의 핵심

- ACID 트랜잭션: 데이터 무결성 보장.
- 스키마 진화: 유연한 스키마 변경.
- 타임 트래블: 과거 데이터 조회/복구.
- 세분화된 DML: DELETE, UPDATE, MERGE 지원.
- 오픈 포맷: Parquet 기반, 벤더 종속성 회피.

스파크 성능 최적화

- DAG: 작업 실행 계획.
- 병목 현상: 스필(메모리), 셔플(네트워크), 스큐(부하 불균형), 작은 파일 문제.
- 해결: 메모리 증설, 셔플 최소화, 파티셔닝 최적화, 파일 압축.

메달리온 아키텍처

브론즈 (원본) → 실버 (정제) → 골드 (분석/ML)

Databricks 데이터 수집

- 오토로더: 증분/스트리밍 파일 수집.
- 카피 인투: 멍등성 배치 수집.
- DLT: 선언적 파이프라인, 품질 관리.

데이터 거버넌스

- 필수 요소: 발견 가능성, 접근 제어, 품질, 계보, 감사.
- 도구: 유니티 카탈로그 (Unity Catalog).

Databricks Jobs (워크플로우)

- **DAG 기반:** 태스크, 의존성, 스케줄링.
- **자동화:** 재시도, 알림, 수리 후 실행.
- **파라미터:** 잡/태스크 파라미터 전달.

데이터 아키텍처 패러다임

- **데이터 메시:** 도메인 소유권, 데이터를 제품으로, 셀프 서비스, 연합 거버넌스 (조직/문화).
- **데이터 패브릭:** 기술 중심, 통합 및 자동화, 단일 환경처럼 제공 (기술/아키텍처).

9 체크리스트: 학습 및 과제 대비

CSCI103 Lecture 5 학습 체크리스트

- 핵심 개념 이해

데이터 사일로, 데이터 레이크, 데이터 스웸프, 데이터 웨어하우스, 레이크하우스의 정의와 차이점을 설명할 수 있는가?

스키마 온 리드와 스키마 온 라이트의 개념과 각각의 장단점을 아는가?

델타 레이크의 주요 기능(ACID, 스키마 진화, 타임 트래블, DML)을 설명하고 왜 필요한지 아는가?

스파크의 DAG, 잡, 스테이지, 태스크의 관계를 이해하는가?

스파크 성능 병목 현상(스필, 셔플, 스큐, 작은 파일)의 원인과 해결 전략을 아는가?

메달리온 아키텍처(브론즈, 실버, 골드)의 각 레이어 역할과 목적을 아는가?

데이터 거버넌스의 중요성과 주요 구성 요소(발견 가능성, 접근 제어, 품질, 계보)를 아는가?

데이터 메시와 데이터 패브릭의 핵심 아이디어와 차이점을 설명할 수 있는가?

- Databricks 도구 활용

오토로더와 카피 인투의 사용 목적과 차이점을 아는가?

Databricks Jobs를 사용하여 워크플로우를 구성하고 태스크 의존성을 설정할 수 있는가?

잡 스케줄링(크론, 파일 트리거, 연속) 옵션을 이해하는가?

'수리 후 실행(Repair and Run)' 기능의 이점을 설명할 수 있는가?

델타 테이블에 대한 SQL DML(INSERT, DELETE, UPDATE, MERGE) 작업을 수행할 수 있는가?

델타 테이블의 타임 트래블 기능을 사용하여 과거 데이터를 조회할 수 있는가?

- 과제 대비

다음 과제(Assignment 3)에서 스트리밍 및 SCD Type 1/Type 2 구현이 필요함을 인지하고 있는가?

델타 레이크의 증분 수집 및 스키마 진화 기능이 과제에 어떻게 적용될지 예상할 수 있는가?